Submission Worksheet

CLICK TO GRADE

https://learn.ethereallab.app/assignment/IT114-006-S2024/it114-project-milestone-1/grade/fm362

IT114-006-S2024 - [IT114] Project Milestone 1

Submissions:

Submission Selection

1 Submission [active] 3/18/2024 3:21:48 PM

Instructions

COLLAPSE

Create a new branch called Milestone1
At the root of your repository create a folder called Project if one doesn't exist yet
    You will be updating this folder with new code as you do milestones
    You won't be creating separate folders for milestones; milestones are just branches
Create a pull request from Milestone1 to main (don't complete/merge it yet, just have it in open status)
Copy in the latest Socket sample code from the most recent Socket Part example of the lessons
    Recommended Part 5 (clients should be having names at this point and not ids)
    https://github.com/MattToegel/IT114/tree/Module5/Module5
Fix the package references at the top of each file (these are the only edits you should do at this point)
Git add/commit the baseline and push it to github
Create a pull request from Milestone1 to main (don't complete/merge it yet, just have it in open status)
Ensure the sample is working and fill in the below deliverables
    Note: The client commands likely are different in part 5 with the /name and /connect options instead of just "connect"
Generate the worksheet output file once done and add it to your local repository
Git add/commit/push all changes
Complete the pull request merge from step 7
Locally checkout main
git pull origin main

Branch name: Milestone1

Tasks: 9 Points: 10.00

Start Up (3 pts.)

COLLAPSE

COLLAPSE

Task #1 - Points: 1

Text: Server and Client Initialization

Checklist

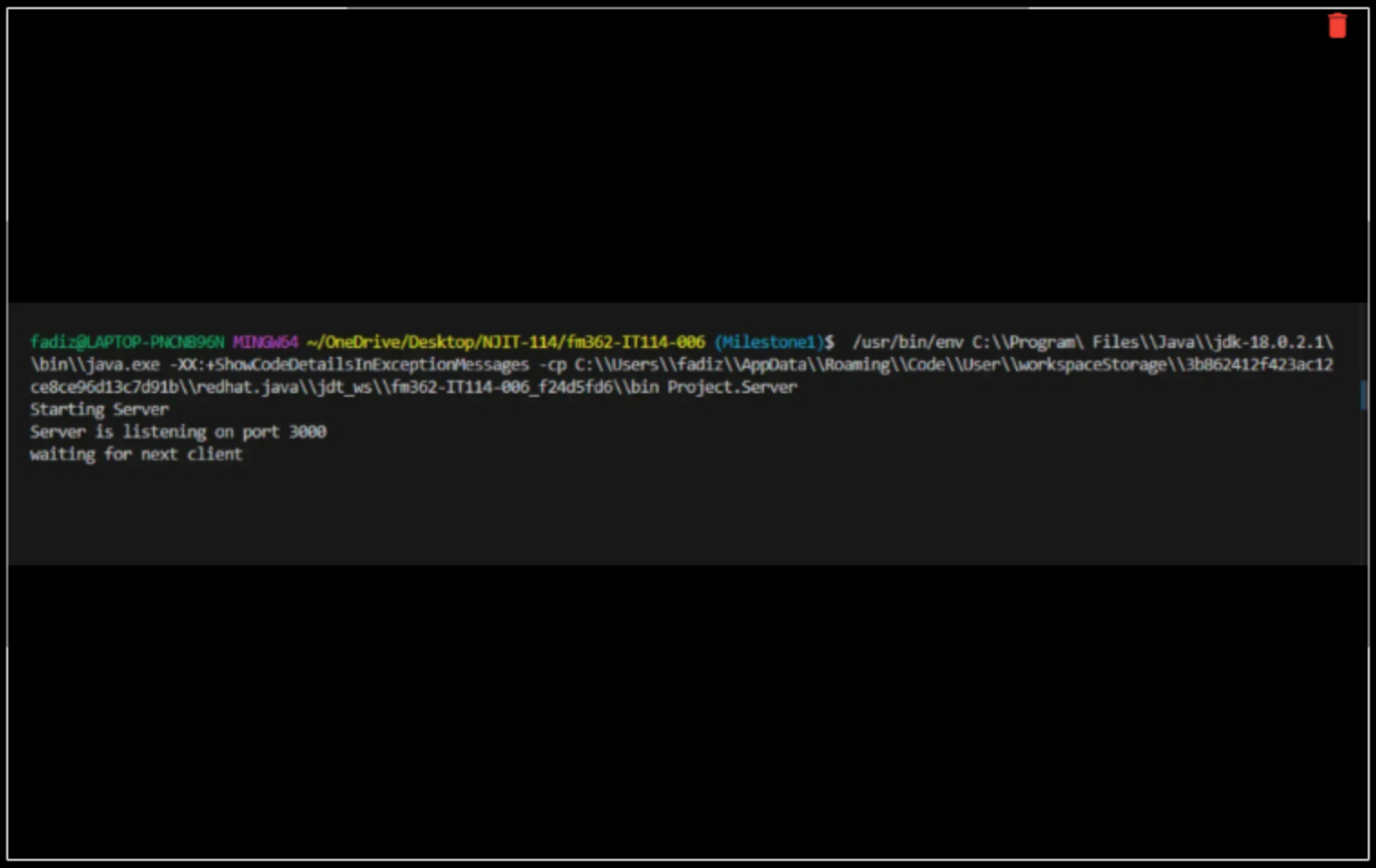
*The checkboxes are for your own tracking

#	Points	Details
#1	1	Server should properly be listening to its port from the command line (note the related message)
#2	1	Clients should be successfully waiting for input
#3	1	Clients should have a name and successfully connected to the server (note related messages)

Task Screenshots:

Gallery Style: Large View

Small Medium Large



The picture shows the server output.

Checklist Items (1)

#1 Server should properly be listening to its port from the command line (note the related message)

```
fadiz@LAPTOP-PNCNB96N MINGW64 ~/OneDrive/Desktop/NJIT-114/fm362-IT114-006 (Milestone1)$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-18.0.2.1\
\bin\\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\fadiz\\AppData\\Roaming\\Code\\User\\workspaceStorage\\3b862412f423ac12
ce8ce96d13c7d91b\\redhat.java\\jdt_ws\\fm362-IT114-006_f24d5fd6\\bin Project.Client

Listening for input
Waiting for input
```

Run: Server
bash
Run: Client

Client server output is waiting for an input.

Checklist Items (1)

#2 Clients should be successfully waiting for input

PROBLEMS 16 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
$ javac Project/Server.java

fadiz@LAPTOP-PNCNB96N MINGW64 ~/OneDrive/Des
ktop/NJIT-114/fm362-IT114-006 (Milestone1)
$ java Project.Server
Starting Server
Server is listening on port 3000
waiting for next client
waiting for next client
Client connected
Thread[14]: Thread created
Thread[14]: Thread starting
Thread-0 leaving room Lobby
Thread-0 joining room Lobby
Thread[14]: Received from client: Type[CONNE
CT], Number[0], Message[null]
```

```
/connect localhost:3000
You must set your name before you can connec
t via: /name your_name
Waiting for input
/name Fadi
Name set to Fadi
Waiting for input
Hello
Not connected to server
Waiting for input
/connect localhost:3000
Client connected
Waiting for input
Debug Info: Type[CONNECT], Number[0], Messag
e[connected]
*Fadi connected*
```

Right terminal is connected to the server and a name is set as Fadi. Left terminal shows that the client is successfully connected.

Checklist Items (1)

#3 Clients should have a name and successfully connected to the server (note related messages)

COLLAPSE

Task #2 - Points: 1

Text: Explain the connection process

Details:

Note the various steps from the beginning to when the client is fully connected and able to communicate in the room.

Emphasize the code flow and the sockets usage.

Checklist

*The checkboxes are for your own tracking

#	Points	Details
#1	1	Mention how the server-side of the connection works
#2	1	Mention how the client-side of the connection works
#3	1	Describe the socket steps until the server is waiting for messages from the client

Response:

From the server-side the server determines and starts listening to the port for an incoming connections. it creates a socket using port number, the server keeps accepting incoming clients. Client can connect to the server using the port number, it creates a client socket and begins the with the server. if the connection is successful the client can send and receive messages, and the client may ask for name when connecting to the server. the socket steps until the server is waiting for messages is the server creates a socket and attach a port number and the rest of the steps are exactly as I mentioned lately in this paragraph.

COLLAPSE

Communication (3 pts.)

COLLAPSE

Task #1 - Points: 1

Text: Add screenshot(s) showing evidence related to the checklist

Checklist

*The checkboxes are for your own tracking

#	Points	Details
#1	1	At least two clients connected to the server
	1	Client can send messages to the server

#2		
#3	1	Server sends the message to all clients in the same room
#4	1	Messages clearly show who the message is from (i.e., client name is clearly with the message)
#5	2	Demonstrate clients in two different rooms can't send/receive messages to each other (clearly show the clients are in different rooms via the commands demonstrated in the lessons
#6	1	Clearly caption each image regarding what is being shown

Task Screenshots:

Gallery Style: Large View

Small Medium Large

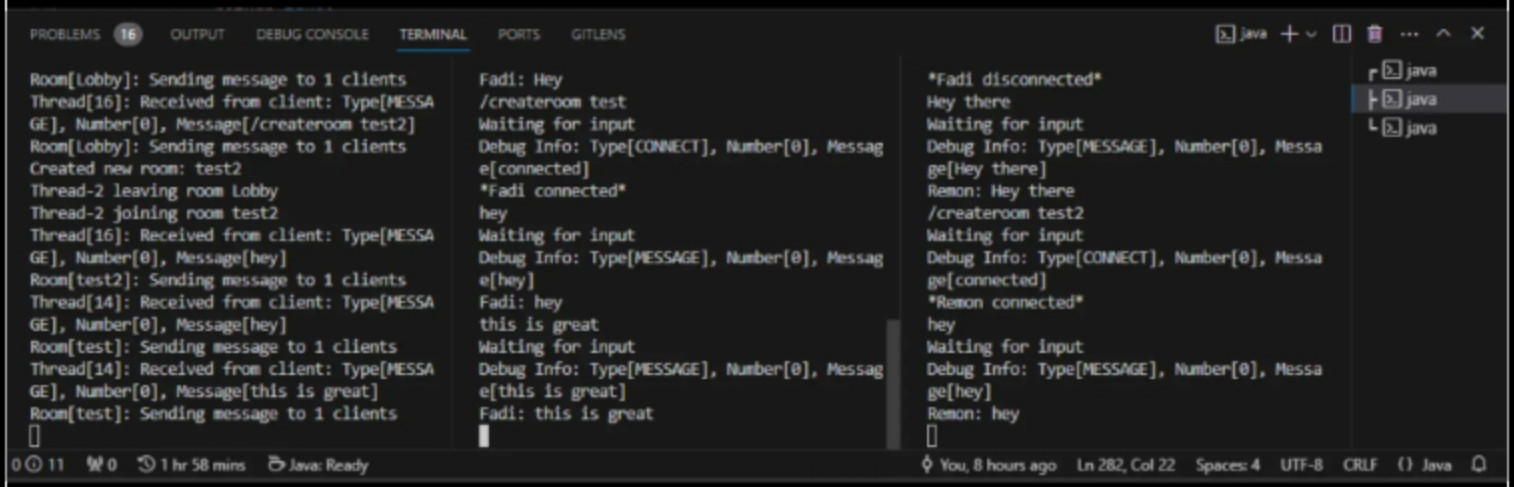
This Picture shows two clients are connected which is Fadi and Remon. And Client Fadi sent a message saying Hey. The message "Hey" show that Fadi sent it.

Checklist Items (3)

#1 At least two clients connected to the server

#2 Client can send messages to the server

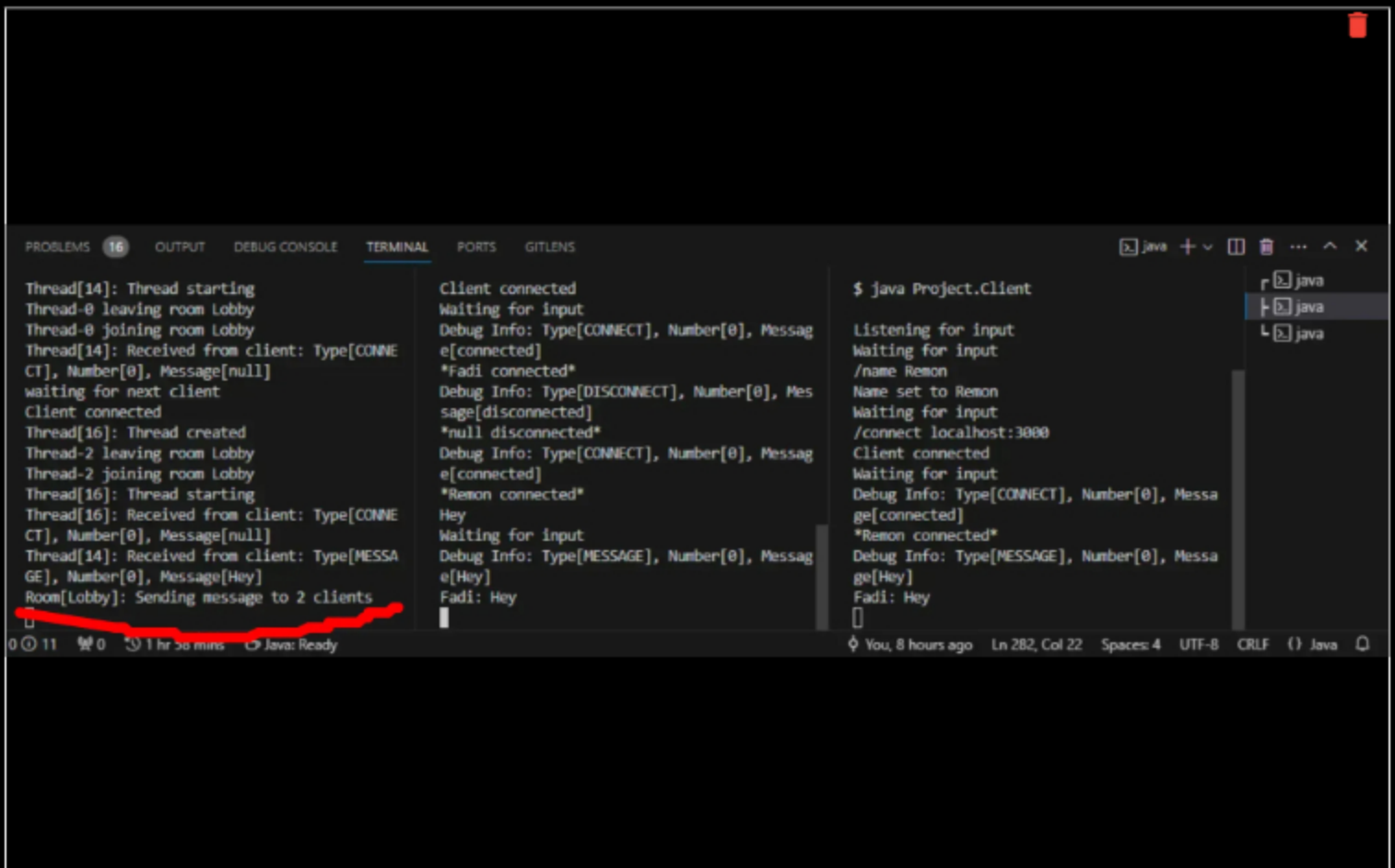
#4 Messages clearly show who the message is from (i.e., client name is clearly with the message)



This picture shows that the two clients are in 2 different rooms and messages are not appearing to each other.

Checklist Items (1)

#5 Demonstrate clients in two different rooms can't send/receive messages to each other (clearly show the clients are in different rooms via the commands demonstrated in the lessons



This picture shows the the server sends the message to all clients.

Checklist Items (1)

#3 Server sends the message to all clients in the same room

COLLAPSE

Task #2 - Points: 1

Text: Explain the communication process

Details:

How are messages entered from the client side and how do they propagate to other clients?

Note all the steps involved and use specific terminology from the code.
Don't just translate the code line-by-line to plain English, keep it concise.

Checklist

*The checkboxes are for your own tracking

#	Points	Details
#1	1	Mention the client-side (sending)
#2	1	Mention the ServerThread's involvement
#3	1	Mention the Room's perspective
#4	1	Mention the client-side (receiving)

Response:

The client side, messages are entered from the client user and then the client sends the message to the server. Server Thread, it shows the communication with a specific client receives the message from client's socket. The room is a chat room where clients can send messages to each other and interact and this where Server Thread takes place. Client's server thread gets the message from the server and the client displays the message that was received.

COLLAPSE

Disconnecting/Termination (3 pts.)

COLLAPSE

Task #1 - Points: 1

Text: Add screenshot(s) showing evidence related to the checklist

Checklist

*The checkboxes are for your own tracking

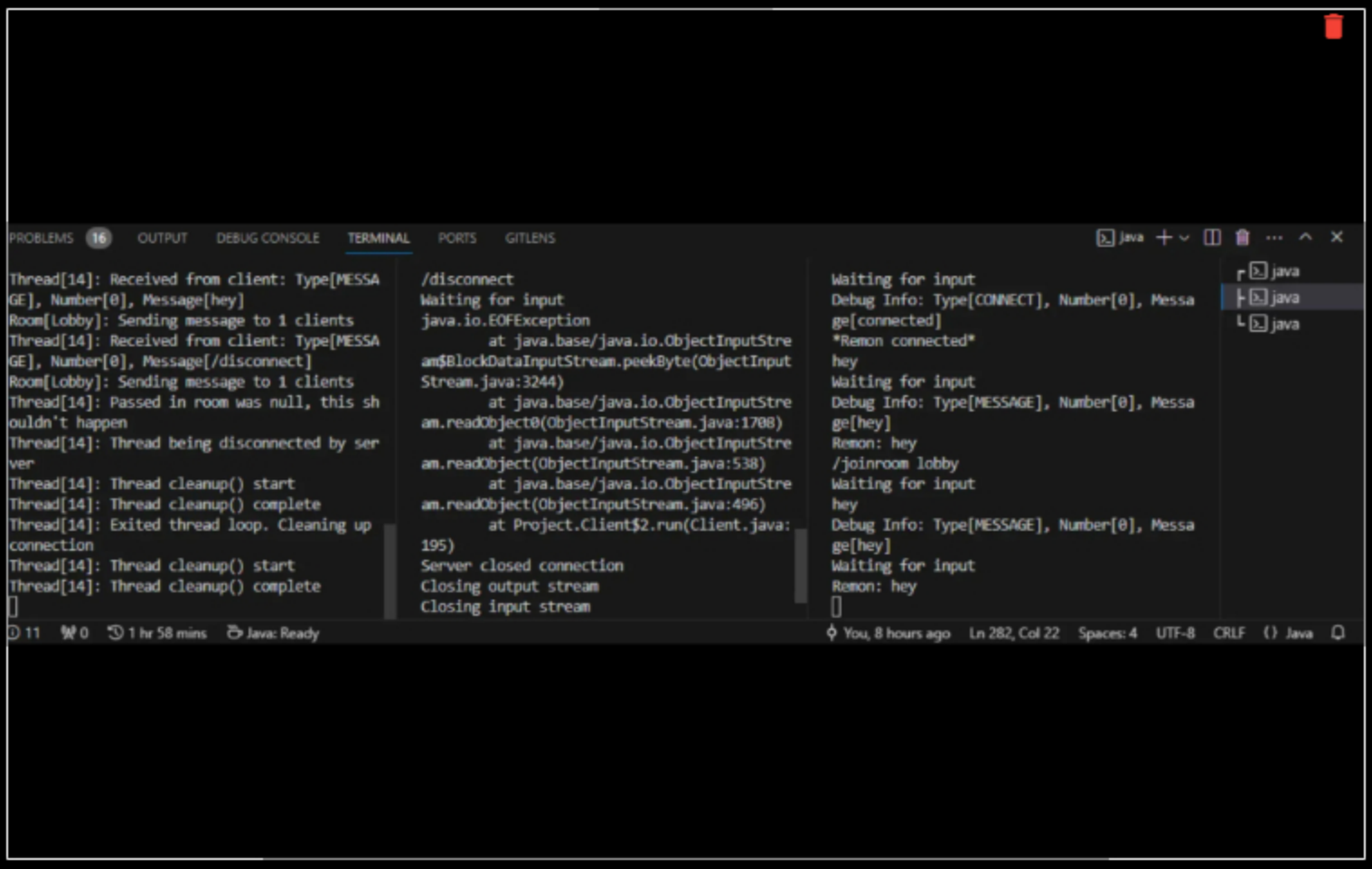
#	Points	Details
#1	1	Show a client disconnecting from the server; Server should still be running without issue (it's ok if an exception message shows as it's part of the lesson code, the server just shouldn't terminate)
		Show the server terminating; Clients should be disconnected but still running and able to reconnect

#2	1	when the server is back online (demonstrate this)
#3	1	For each scenario, disconnected messages should be shown to the clients (should show a different person disconnected and should show the specific client disconnected)
#4	1	Clearly caption each image regarding what is being shown

Task Screenshots:

Gallery Style: Large View

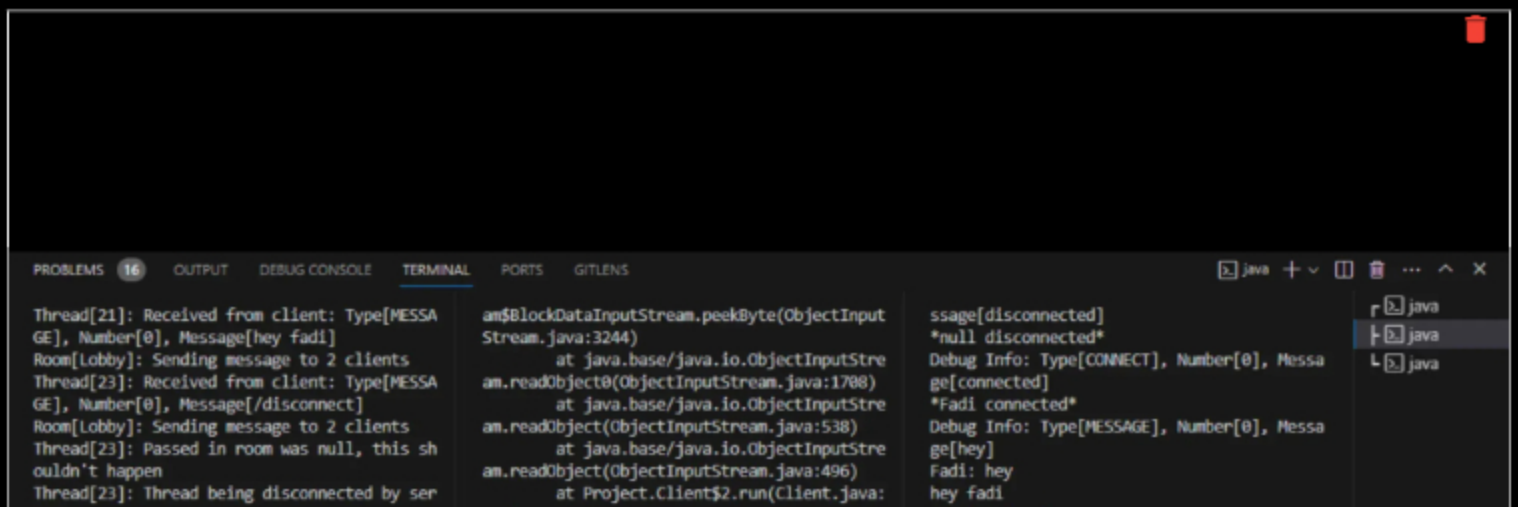
Small Medium Large

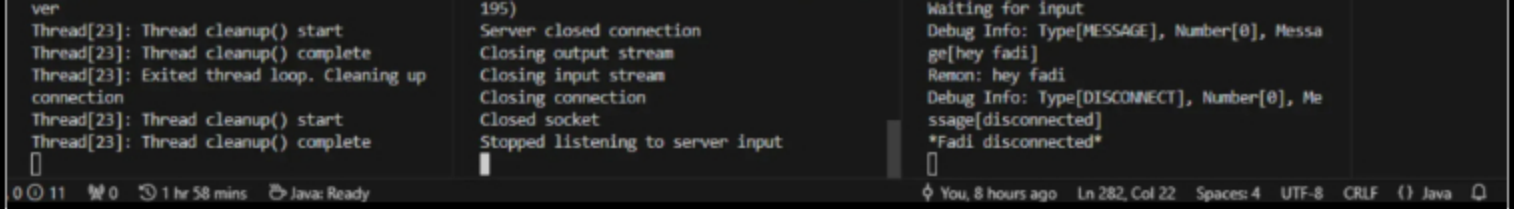


This picture shows that the client in the middle terminal is disconnected.

Checklist Items (1)

#1 Show a client disconnecting from the server; Server should still be running without issue (it's ok if an exception message shows as it's part of the lesson code, the server just shouldn't terminate)





This picture shows that Fadi in the middle terminal disconnected and Remon on the third terminal on the right has received a message that Fadi disconnected. and it also shows on the server terminal that Fadi disconnected and could be able to reconnect.

Checklist Items (2)

#2 Show the server terminating; Clients should be disconnected but still running and able to reconnect when the server is back online (demonstrate this)

#3 For each scenario, disconnected messages should be shown to the clients (should show a different person disconnected and should show the specific client disconnected)

COLLAPSE

Task #2 - Points: 1

Text: Explain the various Disconnect/termination scenarios

Details:
Include the various scenarios of how a disconnect can occur. There should be around 3 or so.

Checklist *The checkboxes are for your own tracking

#	Points	Details
#1	1	Mention how a client gets disconnected from a Socket perspective
#2	1	Mention how/why the client program doesn't crash when the server disconnects/terminates.
#3	1	Mention how the server doesn't crash from the client(s) disconnecting

Response:

A client could get disconnected in a lot of reasons the user can just disconnect from the server by typing /disconnect or a disconnection can happen due to network issue and server termination, The client program is designed to handle these scenarios and it doesn't crash. When a client disconnects the server is designed to handle client connections.

COLLAPSE

Misc (1 pt.)

COLLAPSE

Task #1 - Points: 1

Text: Add the pull request link for this branch

URL #1

https://github.com/Fadimeleika/fm362-IT114-006/pull/9

COLLAPSE

Task #2 - Points: 1

Text: Talk about any issues or learnings during this assignment

Details:

Few related sentences about the Project/sockets topics

Response:

The only issue I had was when I was trying to connect client to the server, It was not letting me until I figured out a typo I was typing connect localhost:3000 instead of /connect localhost:3000.

COLLAPSE

Task #3 - Points: 1

Text: WakaTime Screenshot

Details:

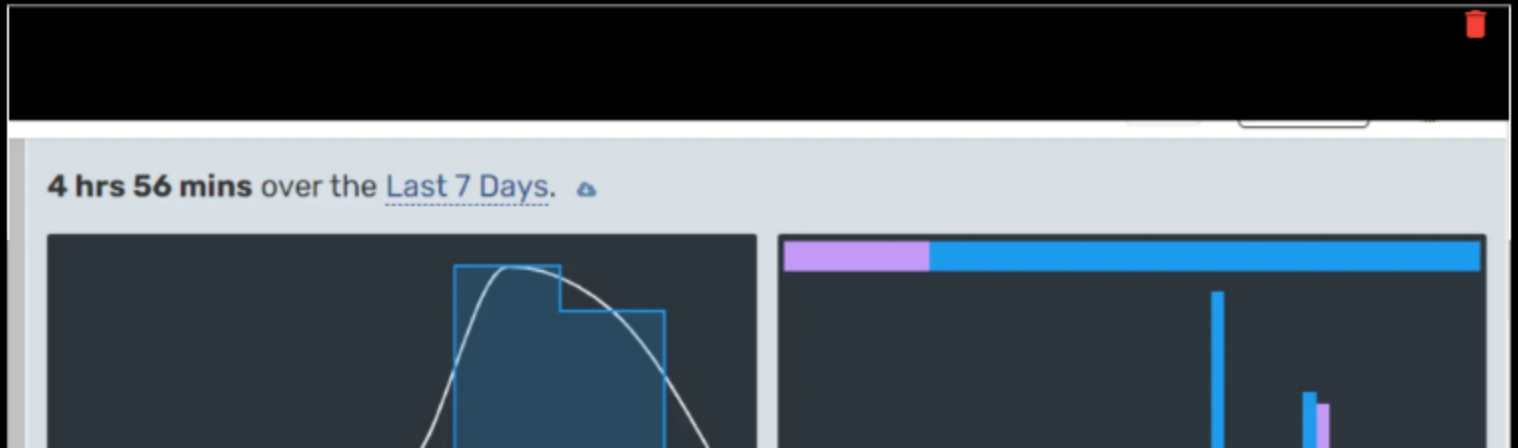
Grab a snippet showing the approximate time involved that clearly shows your repository.

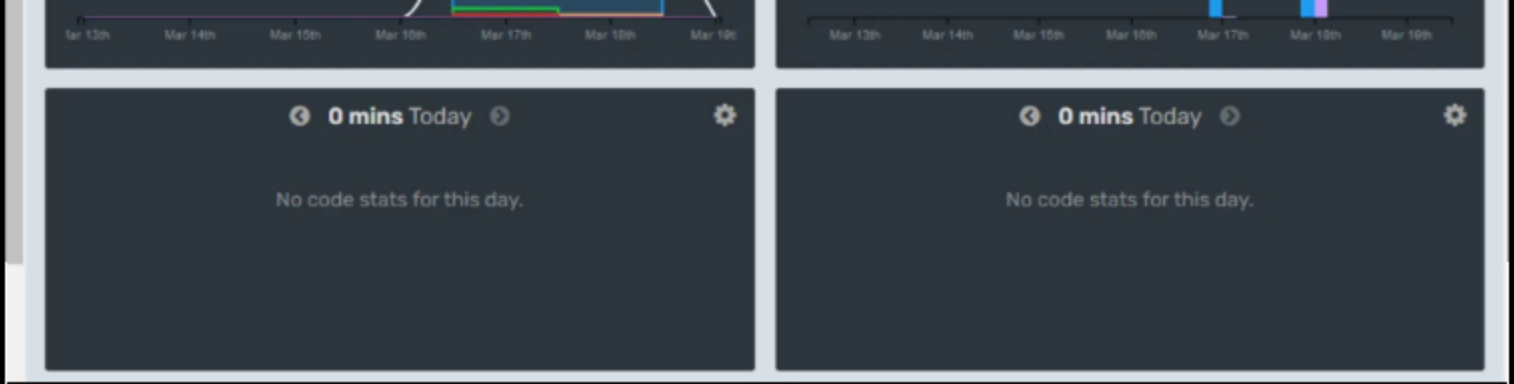
The duration isn't considered for grading, but there should be some time involved.

Task Screenshots:

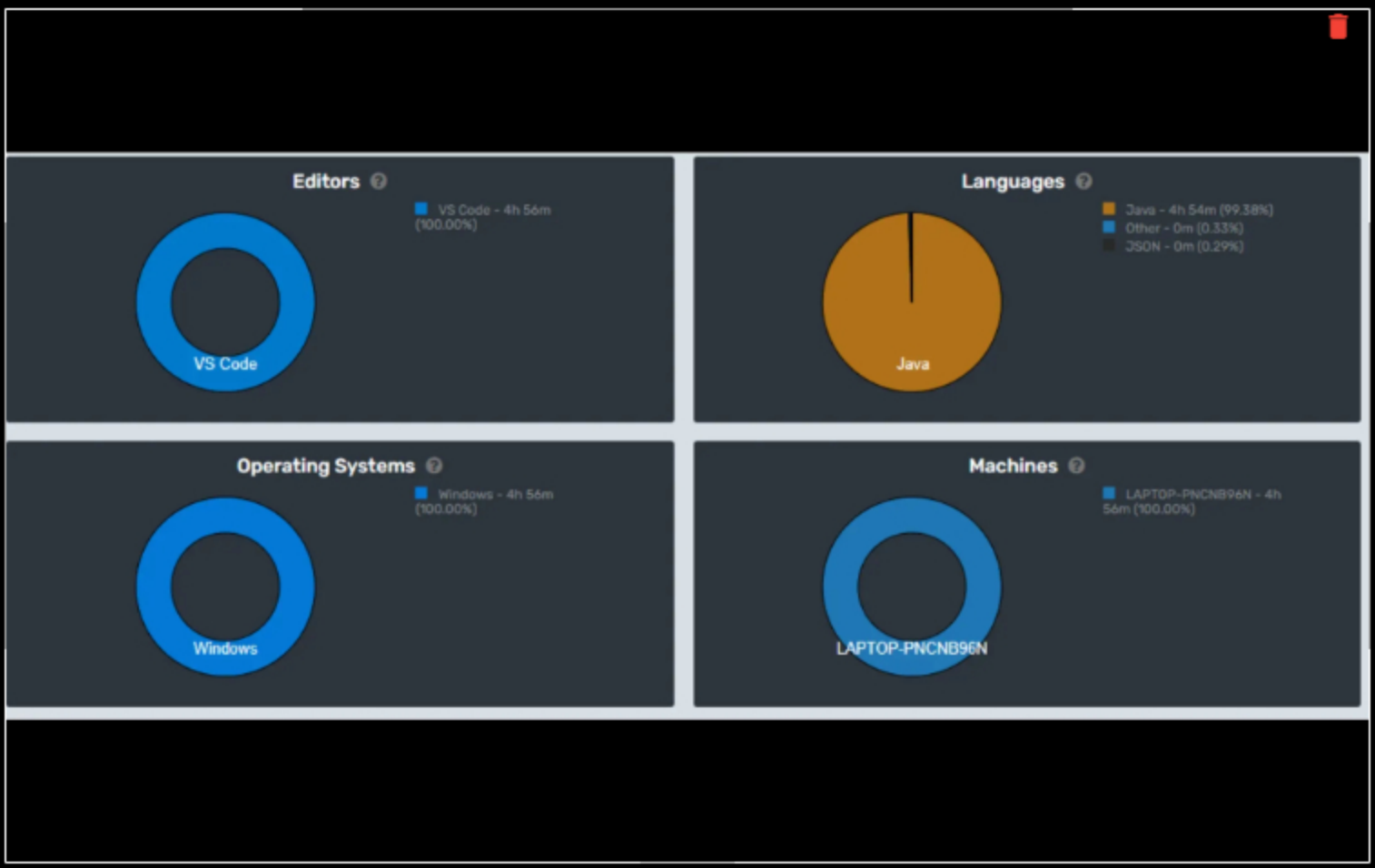
Gallery Style: Large View

Small Medium Large





This is the total time spent.



This is the my machine and the languages that I used

End of Assignment